Traduce Señas:

Manual de instalación

Juan Mauricio Suaza Solórzano

Luis Alejandro Duarte Aldana

Juan Camilo Fierro Esquivel

Instructor: Karol Daniela Correa

4 de noviembre de 2026

Tabla de contenido

Introducción 4

1. Conceptos fundamentales 4

    1.1 Características generales (puertos) 4

    1.2 Base de datos y migraciones 4

    1.3 Variables de configuración (solo nombres) 7

2. Instalación 8

    2.1 Requisitos 8

    2.2 Obtener el código 8

    2.3 Instalación con Docker (ruta recomendada) 9

    2.4 Modo desarrollo sin Docker 10

    2.5 Instalación en servidor de producción 11

3. Configuración y primer uso 12

    3.1 Comprobar que funciona 12

    3.2 Crear la primera cuenta y activarla 13

    3.3 Primer administrador 14

    3.4 Reiniciar la base de datos (borra todos los datos) 14

4. Operación 14

5. Problemas comunes 15

Referencias 17

Introducción

Este manual explica cómo poner Traduce Señas en marcha en un equipo local con Docker Compose, y cómo hacerlo en modo de desarrollo. Está probado con el Backend `v1.4.0` (rama `main`, commit `a962982`) y el Frontend `v1.3.1` (rama `main`, commit `a5e231d`).

No incluye valores de configuración. Las claves y contraseñas las defines tú, en tu archivo `.env`, y **nunca** se escriben en este manual ni en el repositorio.

1. Conceptos fundamentales

1.1 Características generales (puertos)

Tabla 1

Puertos del sistema en el equipo

Componente	Puerto	Publicado en el equipo
api-gateway (único punto de entrada de la API)	8080	Sí. El compose lo publica en todas las interfaces de la máquina.
PostgreSQL de `iam`, `recognition`, `lexicon`, `analytics` y `profile`	5433, 5434, 5435, 5436 y 5437	Solo en `127.0.0.1`
MongoDB (muestras de gestos)	27017	Solo en `127.0.0.1`
Panel de RabbitMQ	15672	Solo en `127.0.0.1`
Servicios `iam`, `recognition`, `lexicon`, `analytics` y `profile` (3001 a 3005) y AMQP (5672)	—	No se publican: solo son accesibles dentro de la red de Docker
Servidor de desarrollo de Expo (web)	8081	No forma parte del compose

Para llamar a la API desde fuera de Docker usa siempre `http://localhost:8080/api/...`. Si la red del equipo no es de confianza, puedes cambiar el puerto del gateway en `docker-compose.yml` a `127.0.0.1:8080:8080`.

1.2 Base de datos y migraciones

Cada servicio tiene su propia base de datos. Los archivos SQL están en `services/<servicio>/db/` y el compose los monta en cada contenedor de base de datos.

PostgreSQL ejecuta esos archivos **solo cuando el volumen de datos se crea por primera vez**. La lista completa de archivos montados está en el Manual técnico, sección 4.2.

Consecuencias prácticas:

- En una instalación nueva no tienes que hacer nada: el esquema se crea solo al primer `docker compose up`.
- Si ya tienes volúmenes creados con una versión anterior, **no los borres** para tener las migraciones nuevas: aplícalas a mano, en orden, con la plataforma levantada (`docker compose up -d`). Los comandos están abajo.
- Si borras los volúmenes con `docker compose down -v`, se pierden todos los datos.

Migraciones para un volumen que ya existe. Ejecútalas desde la raíz del repositorio Backend, en este orden. Cada archivo se ejecuta una sola vez, salvo `009_outbox_events.sql` de `iam`, que es idempotente y puedes repetirla si ya se aplicó con una versión anterior.

```
docker compose exec -T postgres-iam       psql -U postgres -d iam
< services/iam-service/db/009_outbox_events.sql
docker compose exec -T postgres-iam       psql -U postgres -d iam
< services/iam-service/db/010_users_password_nullable.sql
docker compose exec -T postgres-iam       psql -U postgres -d iam
< services/iam-service/db/011_account_deletion_token.sql
docker compose exec -T postgres-iam       psql -U postgres -d iam
< services/iam-service/db/012_outbox_claim_token.sql
docker compose exec -T postgres-iam       psql -U postgres -d iam
< services/iam-service/db/013_users_terms_version.sql
docker compose exec -T postgres-analytics psql -U postgres -d analytics
< services/analytics-service/db/002_erased_users.sql
docker compose exec -T postgres-profile   psql -U postgres -d profile
< services/profile-service/db/002_erased_users.sql
docker compose exec -T postgres-profile   psql -U postgres -d profile
< services/profile-service/db/003_ui_languages.sql
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-
service/db/004_achievement_catalog.sql
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-
service/db/005_obtainable_achievements.sql
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-
service/db/006_translation_milestones.sql
```

```
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-
service/db/007_functional_achievements.sql
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-service/db/008_admin_reports.sql
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-
service/db/009_daily_challenges.sql
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-
service/db/010_challenge_reward_achievements.sql
docker compose exec -T postgres-recognition psql -U postgres -d
recognition < services/recognition-service/db/003_erased_users.sql
docker compose exec -T postgres-recognition psql -X -v ON_ERROR_STOP=1
-U postgres -d recognition < services/recognition-
service/db/004_sign_templates_word_kind.sql
docker compose exec -T postgres-recognition psql -X -v ON_ERROR_STOP=1
-U postgres -d recognition < services/recognition-
service/db/005_translation_retention.sql
```

Notas sobre esta lista:

- Esta lista es la completa. El comentario del principio de `docker-compose.yml` omite `iam-service/db/012_outbox_claim_token.sql` y `profile-service/db/007_functional_achievements.sql`, aunque ambos archivos están montados.
- Si usas PowerShell, el operador `<` no funciona. Sustitúyelo por una tubería, por ejemplo: `Get-Content services/iam-service/db/009_outbox_events.sql | docker compose exec -T postgres-iam psql -U postgres -d iam`.
- Las migraciones 009 y 010 de `profile` añaden los retos diarios y las medallas por puntos. Antes de aplicarlas, detén el servicio con `docker compose stop profile-service` y vuelve a arrancarlo con `docker compose up -d profile-service` cuando termines.
- Respeta el orden de la lista: los archivos de `iam` dependen de los anteriores, y `009_outbox_events.sql` debe existir antes de que `iam` publique `iam.UserDeleted`.

1.3 Variables de configuración (solo nombres)

El archivo `.env.example` de la raíz del repositorio Backend documenta todas las variables. Para Docker Compose, copia ese archivo a `.env` en la raíz del Backend.

Tabla 2

Variables de configuración (solo nombres)

Variable	Obligatoria	Para qué
`JWT_SECRET`	Sí	Secreto compartido para firmar y verificar las sesiones. Mínimo 32 caracteres. Todos los servicios usan el mismo.
`POSTGRES_PASSWORD`	Sí	Contraseña de los cinco PostgreSQL (usuario `postgres`).
`MONGO_ROOT_PASSWORD`	Sí	Contraseña del usuario de MongoDB.
`RABBITMQ_PASSWORD`	Sí	Contraseña del usuario de RabbitMQ.
`MONGO_ROOT_USER`, `RABBITMQ_USER`	No	Nombres de esos usuarios (por defecto `signa`).
`CORS_ORIGINS`	No	Orígenes web (separados por comas) que pueden llamar a la API. Por defecto, Expo web en local.
`EMAIL_USER`, `EMAIL_PASS`	No	Cuenta de Gmail que envía los códigos y los avisos por correo. Usa una contraseña de aplicación.
`GOOGLE_CLIENT_IDS`	No	Identificadores de cliente de OAuth de Google. Si está vacía, el inicio con Google responde 503.
`FACEBOOK_APP_ID`, `FACEBOOK_APP_SECRET`	No	Datos de la aplicación de Facebook. Si están vacíos, el inicio con Facebook responde 503.
`RECOGNITION_TRANSLATION_RETENTION_DAYS`	No	Días que se guardan las traducciones. Por defecto, 30. Con `0` no se borran.
`ADMIN_REPORTS_ENABLED`, `APP_URL`	No	Activa los reportes por correo a administradores y fija la dirección del botón de esos correos.
`DOMAIN`, `FRONTEND_DIR`, `CORS_ORIGINS_PROD`, `GATEWAY_TRUST_PROXY`	No	Solo para producción (ver la sección 2.5).

Las credenciales de las bases de datos no están en `.env`: el compose las fija a partir de `POSTGRES_PASSWORD`. El frontend tiene su propia variable, `EXPO_PUBLIC_API_URL`, que se explica en la sección 2.3.

2. Instalación

2.1 Requisitos

Tabla 3

Requisitos

Necesitas	Detalle
Git	Para clonar los repositorios.
Docker Desktop	Con `docker` y `docker compose` disponibles (Docker 24 o superior y Compose 2.20 o superior, según la guía técnica del equipo).
Node.js 20 o superior	Solo para el modo desarrollo sin Docker (sección 2.4) y para el frontend (`npm install`).
Una terminal	PowerShell o Git Bash. Los comandos de la sección 1.2 con < se ejecutan en Git Bash.
Espacio en disco	Varios GB libres para imágenes de Docker y dependencias.
Internet	Para descargar imágenes y dependencias, y para que el navegador baje MediaPipe al usar la cámara.
Una cuenta de correo para enviar códigos	Opcional. Sin ella, el registro funciona, pero el código no llega por correo (ver la sección 3.2).
Expo Go en el celular	Opcional, para probar la app en un teléfono.

2.2 Obtener el código

Hay dos repositorios públicos, ambos en la rama `main`:

```
git clone -b main https://github.com/mauro-1508/backendTS.git
git clone -b main https://github.com/tadeo77789/frontendTS.git
```

Confirma las versiones con `git -C backendTS rev-parse --short HEAD` (debe mostrar `a962982`) y `git -C frontendTS rev-parse --short HEAD` (debe mostrar `a5e231d`). Si `main` ya avanzó, usa la versión que indique el Manual técnico.

El repositorio Backend contiene `docker-compose.yml`, `docker-compose.prod.yml`, `.env.example` y los Dockerfile de cada servicio. No necesitas ningún archivo de otro lugar.

2.3 Instalación con Docker (ruta recomendada)

Paso A. Crea el archivo de configuración. Desde la raíz del repositorio Backend:

```
cd backendTS
cp .env.example .env
```

En PowerShell, usa `Copy-Item .env.example .env`. Abre `.env` y rellena, como mínimo:

- `JWT_SECRET`: texto de 32 caracteres o más. Puedes generarlo con `openssl rand -hex 32`.
- `POSTGRES_PASSWORD`, `MONGO_ROOT_PASSWORD` y `RABBITMQ_PASSWORD`: contraseñas tuyas. No uses los caracteres `@ : / ? #`, porque van dentro de las URL de conexión.

Si quieres que lleguen los códigos por correo, rellena también `EMAIL_USER` y `EMAIL_PASS`. No subas `.env` a ningún repositorio.

Paso B. Construye y levanta la plataforma:

```
docker compose up --build -d
docker compose ps
```

La primera vez tarda varios minutos, porque construye las imágenes. El compose levanta 13 contenedores: el gateway, los cinco servicios, los cinco PostgreSQL, MongoDB y RabbitMQ. Cada servicio espera a que su base de datos esté sana, y el gateway espera a que los cinco servicios respondan. Repite `docker compose ps` hasta que todos aparezcan como `running` o `healthy`.

Para apagar sin borrar datos: `docker compose down`. Para borrar también los datos: `docker compose down -v` (ver la sección 3.4).

Paso C. Levanta el frontend. En otra terminal, desde la carpeta del Backend (la que contiene `frontendTS`):

```
cd ../frontendTS
npm install
```

Crea un archivo `.env` en la raíz del frontend con esta línea:

```
EXPO_PUBLIC_API_URL=http://localhost:8080/api
```

Para probar desde un celular en la misma red, usa la IP de tu equipo en lugar de `localhost` (por ejemplo, `http://192.168.1.20:8080/api`). La variable se lee al compilar: si la cambias, reinicia Expo.

```
npm run web
```

Expo abre la web en `http://localhost:8081`. El backend ya acepta ese origen por defecto. Para la app móvil usa `npm start` y Expo Go.

2.4 Modo desarrollo sin Docker

Sirve para programar con recarga automática. Necesitas Node.js 20 o superior y las bases de datos de Docker, que es lo más simple porque el compose ya crea su esquema.

1. **Bases de datos.** Desde la raíz del Backend, levanta solo los PostgreSQL:

   ```
   docker compose up -d postgres-iam postgres-recognition postgres-lexicon
   postgres-analytics postgres-profile
   ```

   Quedan en `127.0.0.1` en los puertos 5433 a 5437.

2. **Dependencias.** En la raíz del Backend: `npm install`.

3. **Configuración de cada servicio.** Copia el `.env.example` de cada servicio a su `.env` (`services/api-gateway`, `services/iam-service`, `services/recognition-service`, `services/lexicon-service`, `services/analytics-service` y `services/profile-service`). Dentro de cada archivo, las variables llevan el nombre sin prefijo: `IAM_DB_HOST` se convierte en `DB_HOST`. Usa el mismo `JWT_SECRET` en todos y, en

`DB_PASSWORD`, el mismo valor de `POSTGRES_PASSWORD`. Con `RABBITMQ_URL` y `MONGO_URL` vacíos, los eventos y las muestras se guardan en memoria: se pierden al reiniciar.

4. **Arranque.** Desde la raíz del Backend: `npm run dev:all`. Levanta el gateway y los cinco servicios con recarga automática. Para un solo servicio: `npm run dev -w @traduce/iam-service`.

Si el stack completo de Docker sigue levantado, el puerto 8080 está ocupado por el gateway del contenedor. Detenlo con `docker compose stop api-gateway`.

2.5 Instalación en servidor de producción

El compose de producción se suma al base con `-f`. Desde la raíz del Backend:

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

Requisitos y variables adicionales:

- El repositorio Frontend clonado junto al Backend, o la ruta indicada en `FRONTEND_DIR` (por defecto `../frontendTS`).
- `DOMAIN` (por defecto `traducesenas.tech`), con el DNS apuntando al servidor, y `www` redirigido al dominio.
- Puertos 80 y 443 abiertos, también en el panel del proveedor. Caddy necesita el 80 para obtener el certificado.
- `CORS_ORIGINS_PROD` solo si necesitas permitir más orígenes web.

Con este archivo el gateway deja de publicarse en el 8080: todo entra por HTTPS. El procedimiento completo de despliegue (respaldos, pruebas de humo y reversión) está en el Plan de despliegue (evidencia 07).

Estado del servidor de producción: comprobado el 4 de noviembre de 2026: HTTPS activo, HTTP redirige a HTTPS y `/api/health` responde `ok` con la versión 1.4.0 y los cinco servicios arriba.

3. Configuración y primer uso

3.1 Comprobar que funciona

1. **Estado de los contenedores.** `docker compose ps`. Los cinco servicios, el gateway y las bases de datos deben aparecer como `running` o `healthy`.
2. **Estado agregado.** Desde el equipo:

   ```
   curl http://localhost:8080/health
   ```

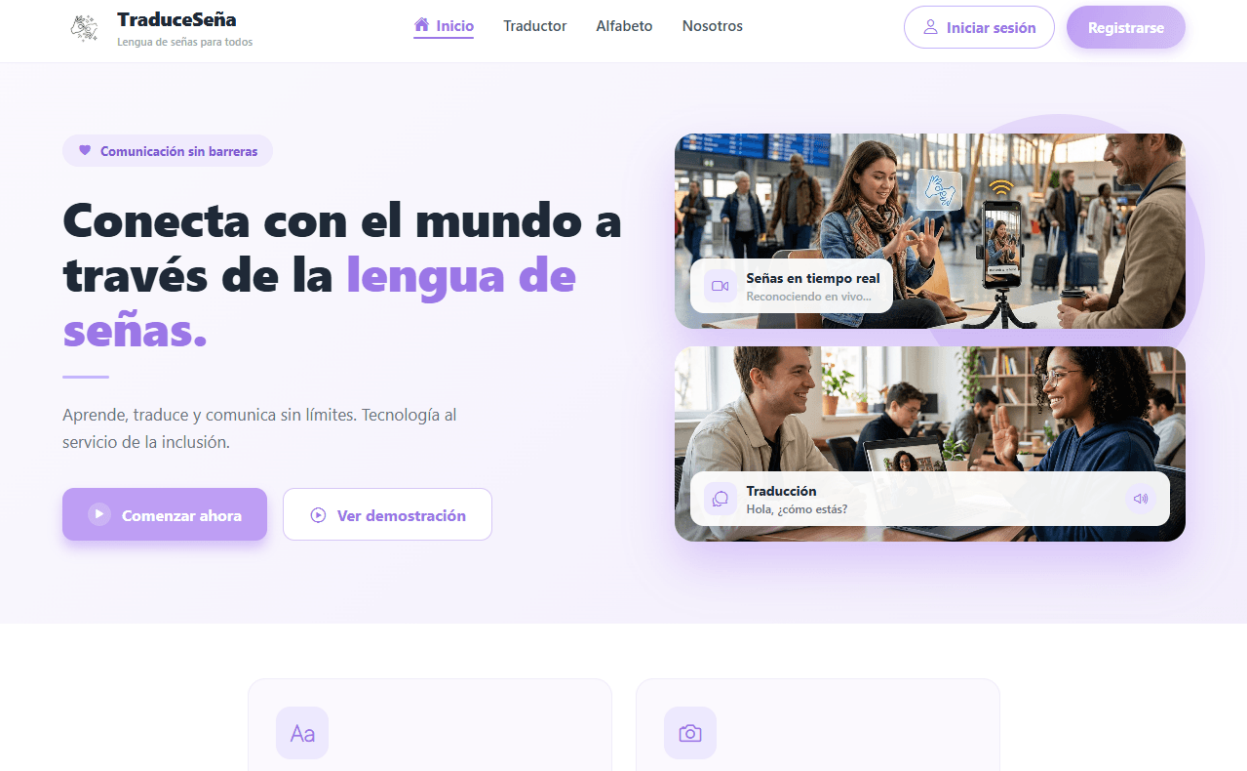
   La respuesta es HTTP 200 con `"status":"ok"` cuando todos los servicios responden. Si uno está caído, el gateway sigue respondiendo 200, pero con `"status":"degraded"` y `"down"` en ese servicio. Al arrancar por primera vez es normal ver `degraded` durante unos segundos: repite la comprobación.
3. **Prueba funcional.** El gateway reenvía la petición a `iam` y la valida antes de hacer nada:

   ```
   curl -i -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json" -d "{}"
   ```

   Debe responder `400` con `"code":"INVALID_BODY"`. Esa respuesta prueba que la petición llegó a `iam`.
4. **Desde la web.** Abre `http://localhost:8081` (con `npm run web`). Debe cargar la página de inicio.

Figura 1

Página de inicio de la aplicación



5. Si algo falla: `docker compose logs <servicio>`, por ejemplo `docker compose logs iam-service`.

3.2 Crear la primera cuenta y activarla

1. En la web, pulsa **Registrarse** y completa el formulario **Crea tu Cuenta**. Acepta los términos.
2. Las cuentas nuevas quedan **inactivas** hasta confirmar el correo con un código. Si configuraste `EMAIL_USER` y `EMAIL_PASS`, el código llega al correo. Si no, activa la cuenta a mano desde la raíz del Backend:

   ```
   docker compose exec -T postgres-iam psql -U postgres -d iam -c "UPDATE users SET status='ACTIVE' WHERE email='<correo_en_minusculas>'"
   ```

   El correo se guarda en minúsculas, así que escríbelo así en la consulta.

Si el registro dice que no se pudo enviar el código, la cuenta sí se creó: pide un código nuevo con **Reenviar** cuando tengas el correo configurado.

3.3 Primer administrador

Se hace con una orden SQL una sola vez: se asigna el rol `ADMIN` al correo de la cuenta. El procedimiento paso a paso está en el Manual del administrador, sección 3.

3.4 Reiniciar la base de datos (borra todos los datos)

Esto borra TODOS los datos de todas las bases (cuentas, historial, plantillas y muestras). Si te importan, haz una copia antes, por ejemplo con `docker compose exec -T postgres-iam pg_dump -U postgres iam > copia-iam.sql`, y repítela para cada base.

```
docker compose down -v
docker compose up --build -d
```

Al recrear los volúmenes, PostgreSQL vuelve a ejecutar los archivos SQL montados, así que no hace falta aplicar migraciones a mano.

4. Operación

Tabla 4

Operación

Quiero...	Orden
Detener todo sin borrar datos	`docker compose stop`
Volver a encender	`docker compose up -d`
Ver el estado	`docker compose ps`
Detener y quitar contenedores (los datos se conservan)	`docker compose down`
Ver los mensajes de un servicio	`docker compose logs -f iam-service`
Reconstruir tras cambiar código	`docker compose up --build -d`
Borrar también los datos	`docker compose down -v` (**borra los datos**)

Quiero...	Orden
Actualizar el código	`git pull` en `backendTS` y en `frontendTS`, y después `docker compose up --build -d`. Las migraciones nuevas no se aplican solas a una base que ya existe: ver la sección 1.2.

5. Problemas comunes

Tabla 5

Problemas comunes

Síntoma	Causa probable	Qué hacer
`docker compose up` falla con "Define JWT_SECRET", "POSTGRES_PASSWORD", "MONGO_ROOT_PASSWORD" o "RABBITMQ_PASSWORD"	Falta esa variable en `.env`	Copia `.env.example` a `.env` en la raíz del Backend y rellénala
Los servicios no arrancan y `iam` se reinicia	`JWT_SECRET` tiene menos de 32 caracteres	Usa un secreto de 32 caracteres o más
`/health` muestra `degraded` o `down` de forma permanente	Una base no responde, RabbitMQ no está listo, o una contraseña con caracteres especiales rompe la URL	Mira `docker compose logs <servicio>`. Quita `@ : / ? #` de las contraseñas y recrea los volúmenes si la contraseña cambió
Cambié un archivo `.sql` y no pasó nada	PostgreSQL solo ejecuta los SQL al crear el volumen	Aplica la migración a mano (sección 1.2) o usa `docker compose down -v` (borra datos)
No llega el código de verificación	`EMAIL_USER` o `EMAIL_PASS` vacíos o rechazados	Activa la cuenta a mano (sección 3.2) o configura una contraseña de aplicación de Gmail y reinicia `iam-service`
El navegador bloquea las llamadas (CORS)	El origen de tu web no está en `CORS_ORIGINS`	Añádelo a `CORS_ORIGINS` y reinicia el gateway con `docker compose up -d api-gateway`
La app no se conecta a tu backend	Apunta a la API pública por defecto	Define `EXPO_PUBLIC_API_URL` en el `.env` del frontend y reinicia Expo
El inicio con Google o Facebook muestra un aviso de no configurado	Faltan `GOOGLE_CLIENT_IDS` o `FACEBOOK_APP_ID` en el backend, o la credencial de la plataforma en el frontend	Configura los identificadores de ambos lados

Síntoma	Causa probable	Qué hacer
El puerto 8080 o 5433 a 5437 está ocupado	Otro programa o el stack de Docker lo usa	Cierra el otro programa o detén el contenedor (`docker compose stop api-gateway`)
`npm run dev:all` no conecta a la base de datos	Los PostgreSQL de Docker no están levantados	Ejecuta `docker compose up -d postgres-iam postgres-recognition postgres-lexicon postgres-analytics postgres-profile`
Expo no arranca o se queda colgado	Caché corrupta	Ejecuta `npx expo start -c`
Todos los usuarios pierden la sesión al reiniciar	`JWT_SECRET` cambió	Usa un `JWT_SECRET` fijo en `.env`
La cámara no funciona en la web	El navegador no tiene permiso o no es un origen seguro	Acepta el permiso. Algunos navegadores solo permiten la cámara en `localhost` o con HTTPS

Referencias

Caddy Project. (s. f.). *Caddy documentation*. Recuperado el 4 de noviembre de 2026, de https://caddyserver.com/docs/

Docker Inc. (s. f.). *Docker Compose*. Docker Docs. Recuperado el 4 de noviembre de 2026, de https://docs.docker.com/compose/

Expo. (s. f.). *Expo documentation*. Recuperado el 4 de noviembre de 2026, de https://docs.expo.dev/

F5, Inc. (s. f.). *nginx documentation*. Recuperado el 4 de noviembre de 2026, de https://nginx.org/en/docs/

OpenJS Foundation. (s. f.). *Node.js documentation*. Recuperado el 4 de noviembre de 2026, de https://nodejs.org/docs/latest/api/

The PostgreSQL Global Development Group. (s. f.). *PostgreSQL 16 documentation*. Recuperado el 4 de noviembre de 2026, de https://www.postgresql.org/docs/16/